Projeto MayaRpg App

Documentação de Containerização e Preparação para Nuvem (DevOps)

INTEGRANTES DO PROJETO e RA'S

André Ferreira da Silva	-	25027670
Cauan Moreira da Silva Lima	-	25027645
Rogerio Rodrigues Vieira	-	25028005
Vitor de Camargo Natário	-	25027184

São Paulo

2026

Sumário

1. Arquiteturas de Containers e Orquestração 3
2. Automação de Deploy 4
3. Fundamentação Teórica: O paradigma da Containerização 5
3.1. Vantagens da Containerização 5
3.2. Ambiente Tradicional vs. Containerizado 6
3.3. Estratégia de Volumes e Persistência de Dados 6
4. REFERÊNCIAS BIBLIOGRÁFICAS 10

1. Arquiteturas de Containers e Orquestração

Para preparar o Maya RPG para o ambiente de nuvem, adotamos a arquitetura de Containers, garantindo que a aplicação e suas dependências rodem de forma isolada, padronizada e imutável em qualquer servidor.

A infraestrutura foi dividida em três arquivos principais:

- Dockerfile (Backend API): Criado na raiz do projeto, ele utiliza uma abordagem Multi-stage Build. Primeiro, utiliza uma imagem base com o Maven para compilar o código e baixar as dependências. Em seguida, gera uma imagem final extremamente leve contendo apenas a JRE (Java Runtime Environment) estritamente necessária para executar o arquivo .jar. Isso reduz o tamanho da imagem e aumenta a segurança.

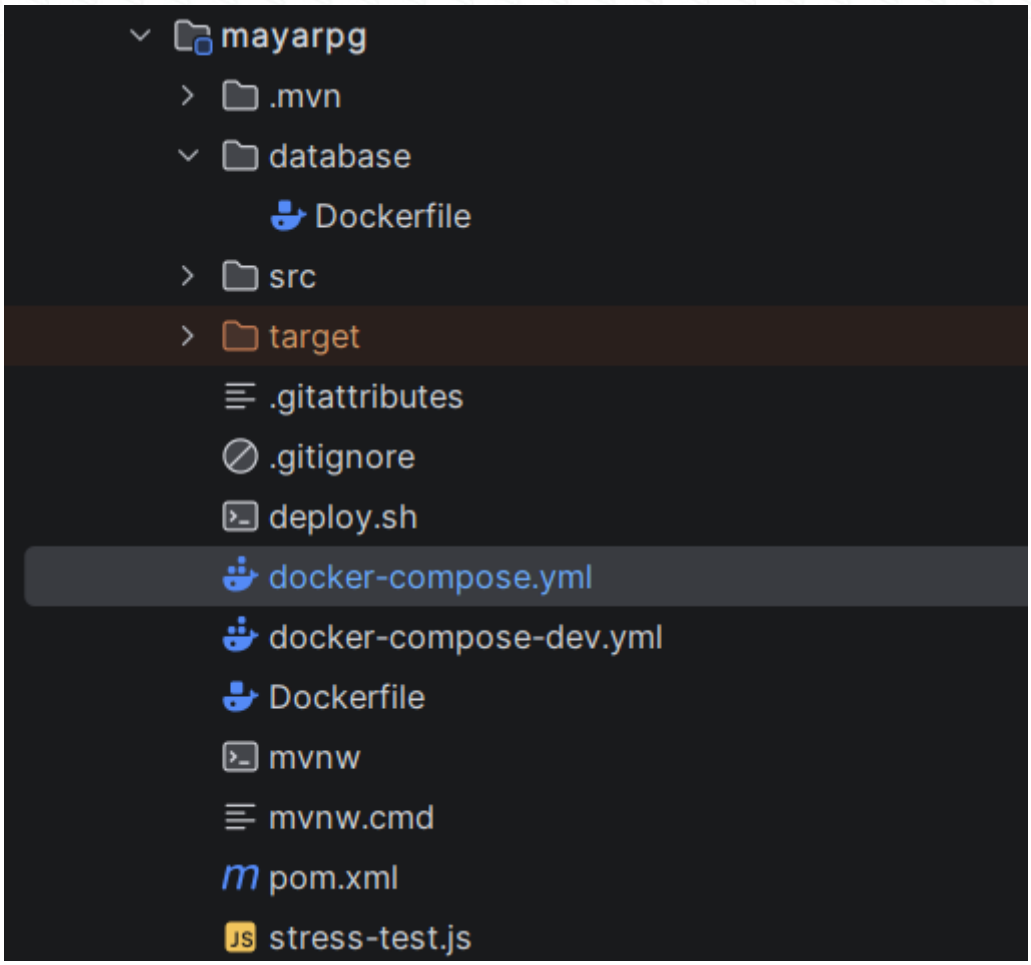
- Dockerfile (Database): Criado na pasta /database, ele consome a imagem oficial postgres:15-alpine. Embora simples, a separação deste arquivo permite futuras customizações e injeções de scripts SQL de inicialização, caso necessário.

- Docker Compose (Orquestração de Ambientes): É o maestro da infraestrutura. Para espelhar o ciclo de vida real de um software, implementamos uma separação clara de ambientes através de dois arquivos distintos::

    1. docker-compose-dev.yml (Ambiente de Desenvolvimento): Otimizado para o dia a dia do programador. Além do banco de dados, ele sobe ferramentas visuais auxiliares (como o pgAdmin), mapeia portas externas específicas (como a porta 5433) para evitar conflitos na máquina local e expõe o banco para testes na IDE.

    2. docker-compose.yml (Ambiente de Produção): Otimizado para o deploy em Nuvem. É enxuto, seguro e orquestra estritamente a API e o PostgreSQL na mesma rede virtual interna. Nele, centralizamos a configuração através de Variáveis de Ambiente (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB), o que é uma prática fundamental de Segurança em Nuvem (Twelve-Factor App), permitindo que credenciais sejam alteradas sem a necessidade de recompilar a aplicação.

Árvore de arquivos do seu projeto:



2. Automação de Deploy

Para mitigar erros humanos durante o processo de atualização da aplicação no servidor em nuvem, desenvolvemos um script de deploy automatizado focado exclusivamente em ambientes Linux/Cloud (deploy.sh).

Fluxo do Script:

1. **Interrupção Segura:** Derruba os containers antigos e libera as portas de rede vinculadas (docker-compose down).

2. **Reconstrução e Inicialização:** Faz o build das novas imagens a partir do código-fonte mais recente e sobe os novos containers em background (docker-compose up -d --build).

3. **Otimização de Recursos:** Realiza a limpeza do servidor, removendo imagens "órfãs" (sistema prune) que ficaram no disco, otimizando o espaço de armazenamento na máquina virtual em nuvem (docker image prune -f).

Imagem do script de deploy:

```
deploy.sh
#!/bin/bash

echo "=========================================="
echo "INICIANDO DEPLOY AUTOMATIZADO - MAYA RPG"
echo "=========================================="

echo -e "\n[1/3] Parando e removendo containers antigos..."
docker-compose down

echo -e "\n[2/3] Construindo novas imagens e subindo o ecossistema..."
docker-compose up -d --build

echo -e "\n[3/3] Limpando imagens soltas (lixo) do Docker..."
docker image prune -f

echo -e "\n==========================================="
echo "DEPLOY FINALIZADO COM SUCESSO!"
echo "A API estara disponivel na porta 8080"
echo "==========================================="
```

3. Fundamentação Teórica: O paradigma da Containerização

A containerização é uma tecnologia de virtualização em nível de sistema operacional que permite empacotar uma aplicação e todas as suas dependências — bibliotecas, frameworks e arquivos de configuração — em uma unidade lógica e padronizada chamada container. Para o projeto Maya RPG, a adoção desse paradigma representa um salto qualitativo na arquitetura, garantindo que o software seja executado de maneira previsível, segura e isolada, independentemente do servidor físico em que for implantado.

3.1. Vantagens da Containerização

A principal vantagem da containerização para o projeto Maya RPG é a Imutabilidade. O container garante que a versão exata do Java, as bibliotecas e o sistema operacional subjacente que rodaram na máquina do desenvolvedor (Local) serão milimetricamente

idênticos aos que rodarão no servidor de Produção (Cloud). Além disso, os containers compartilham o kernel do hospedeiro, tornando-os muito mais rápidos para ligar e desligar do que Máquinas Virtuais tradicionais.

3.2. Ambiente Tradicional vs. Containerizado

No modelo tradicional (Bare Metal ou Virtual Machines), o deploy exigiria acessar o servidor via SSH, instalar o PostgreSQL manualmente, instalar o Java manualmente, configurar portas de firewall e variáveis globais no Linux. Isso gera o clássico problema: "Na minha máquina funciona, mas no servidor não".

No modelo containerizado, a infraestrutura é tratada como código (Infrastructure as Code). O servidor em nuvem só precisa ter o motor do Docker instalado; todo o resto (linguagem, banco, portas internas) é provisionado automaticamente pelo nosso arquivo docker-compose.yml.

3.3. Estratégia de Volumes e Persistência de Dados

Containers são, por natureza, efêmeros. Isso significa que se o container do PostgreSQL for desligado ou deletado pelo nosso script de deploy, todos os dados médicos lá dentro seriam apagados.

Para resolver esse problema crítico, implementamos a estratégia de Docker Volumes Nomeados (postgres_data:/var/lib/postgresql/data). Essa estratégia diz ao Docker para mapear uma pasta segura no disco rígido do servidor físico diretamente para dentro da pasta de dados do container. Dessa forma, mesmo que o container do banco de dados seja completamente destruído, os dados dos pacientes e prontuários permanecem salvos no volume externo, sendo imediatamente reconectados ao novo container assim que ele for instanciado.

Terminal após rodar o comando (docker-compose up -d –build):

```
#25 [api] resolving provenance for metadata file
#25 DONE 0.0s
time="2026-05-06T00:23:13-03:00" level=warning msg="Found orphan con
an run this command with the --remove-orphans flag to clean it up."
[+] up 6/6
 ✓ Image mayarpg-db              Built
 ✓ Image mayarpg-api             Built
 ✓ Volume mayarpg_postgres_data  Created
 ✓ Container mayarpg-db-dev      Recreated
 ✓ Container mayarpg-api         Started
 ✓ Container mayarpg-db          Started
```

Terminal após rodar o comando (docker os) “lista todos os containers em execução”

```
PS C:\Users\andre\OneDrive\Área de Trabalho\clonandoNovo\MayaRpgBackend\mayarpg> docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORTS                                          NAMES
ee0a726945df   mayarpg-api   "java -jar app.jar"      57 seconds ago   Up 56 seconds   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    mayarpg-api
8c15750483d9   mayarpg-db    "docker-entrypoint.s…"   57 seconds ago   Up 56 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    mayarpg-db
```

Conteúdo do Dockerfile do banco de dados presente no caminho “database/Dockerfile”:

Project
database
Dockerfile

Dockerfile

```
# Usa a imagem oficial do PostgreSQL otimizada e leve
FROM postgres:15-alpine
```

Conteúdo do Dockerfile do backend:

Dockerfile

```
# ETAPA 1: Construção (Build)
FROM maven:3.9.6-eclipse-temurin-21-alpine AS builder
WORKDIR /build

COPY pom.xml .
RUN mvn dependency:go-offline

COPY src ./src
RUN mvn clean package -DskipTests

# ETAPA 2: Execução (Run)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

COPY --from=builder /build/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Conteúdo do docker-compose:

docker-compose.yml

```yaml
services:
  # 1. Container do Banco de Dados PostgreSQL
  db:
    build: ./database
    container_name: mayarpg-db
    environment:
      # Aqui o Docker cria o banco, o usuário e dá os acessos automaticamente!
      POSTGRES_DB: mayarpg
      POSTGRES_USER: maya_admin
      POSTGRES_PASSWORD: senha_maya_secreta
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

# 2. Container do backend
  api:
    build: .
    container_name: mayarpg-api
    ports:
      - "8080:8080"
    depends_on:
      - db
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mayarpg
      SPRING_DATASOURCE_USERNAME: maya_admin
      SPRING_DATASOURCE_PASSWORD: senha_maya_secreta

      SPRING_JPA_HIBERNATE_DDL_AUTO: validate
```

Conteúdo do Docker-compose-dev:

docker-compose-dev.yml

```yaml
version: '3.8'

services:
  # Banco de Dados
  db:
    image: postgres:15-alpine
    container_name: mayarpg-db-dev
    environment:
      POSTGRES_DB: mayarpg
      POSTGRES_USER: maya_admin
      POSTGRES_PASSWORD: senha_maya_secreta
    ports:
      - "5433:5432"
    volumes:
      - postgres_data_dev_novo:/var/lib/postgresql/data

  # Interface Gráfica para o Banco
  pgadmin:
    image: dpage/pgadmin4
    container_name: mayarpg-pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80" # Acesse em http://localhost:5050
    depends_on:
      - db

volumes:
  postgres_data_dev_novo:
```

4. Referências Bibliográficas

- **AMAZON WEB SERVICES (AWS).** O que é o Docker?. AWS Brasil. Disponível em: https://aws.amazon.com/pt/docker/.
- **RED HAT.** O que é o Docker? Entenda a tecnologia de containers. Red Hat Brasil. Disponível em: https://www.redhat.com/pt-br/topics/containers/what-is-docker.
- https://fullcycle.com.br/docker-vs-docker-compose/